

SECURE FILE SHARING WITH S3 AND CLOUDFRONT

Services: S3, CloudFront, IAM, Lambda@Edge

Description: Implement a secure file-sharing service where users can upload files to S3, and CloudFront serves them with signed URLs.

Skills: CDN setup, secure file access, IAM policies.

Definition

Secure File Sharing using Amazon S3 and CloudFront is a cloud-based solution that enables controlled access to files stored in Amazon S3. Instead of exposing files publicly, CloudFront distributes the content securely using **signed URLs** that grant temporary access to authorized users.

This architecture ensures that files remain private in S3 while still being delivered efficiently through CloudFront's global content delivery network (CDN). Signed URLs are generated dynamically using AWS Lambda to allow users to access specific files for a limited time.

Scope of the Project

The main objective of this project is to implement a **secure and scalable file-sharing system** using AWS services.

The project focuses on:

- Securely storing files in Amazon S3
- Delivering files through Amazon CloudFront
- Restricting direct access to S3
- Generating temporary signed URLs using AWS Lambda
- Allowing only authorized users to access files

This solution can be used for:

- Secure document sharing
 - Private file distribution
 - Media content delivery
 - Enterprise file access systems
-

Methodologies

The project follows a **secure architecture approach** using AWS best practices.

The implementation methodology includes:

1. Creating a private storage layer using Amazon S3
2. Configuring CloudFront as a secure CDN
3. Restricting direct access using Origin Access Control (OAC)
4. Implementing signed URL authentication
5. Using AWS Lambda to dynamically generate signed URLs
6. Testing and validating secure file access

The system ensures that users can only access files through authenticated requests.

Architecture Flow

Architecture Components

User

↓

Lambda Function (Generates Signed URL)

↓

CloudFront Distribution

↓

Private S3 Bucket

Flow Explanation

1. A user requests access to a file.
2. The request is sent to the backend service (AWS Lambda).
3. Lambda generates a **signed CloudFront URL** with an expiration time.
4. The user receives the signed URL.
5. The user accesses the file using the signed URL.
6. CloudFront verifies the signature.
7. If valid, CloudFront retrieves the file from the private S3 bucket.
8. The file is securely delivered to the user.

Amazon S3

Purpose:

- ## Amazon CloudFront

Purpose:

- Faster file delivery
- Secure content distribution

- Support for signed URLs
 - Integration with private S3 buckets
-

Origin Access Control (OAC)

OAC ensures that CloudFront is the only service allowed to access the S3 bucket.

Purpose:

- Prevent direct public access to S3
 - Allow CloudFront-only access
 - Improve security of stored files
-

AWS Lambda

AWS Lambda is used to generate signed URLs dynamically.

Purpose:

- Serverless backend
 - Generates temporary signed URLs
 - Handles authentication logic
-

CloudFront Key Groups and Public Keys

CloudFront uses public/private key cryptography to validate signed URLs.

Purpose:

- Verify the authenticity of signed URLs
 - Ensure secure access to files
-

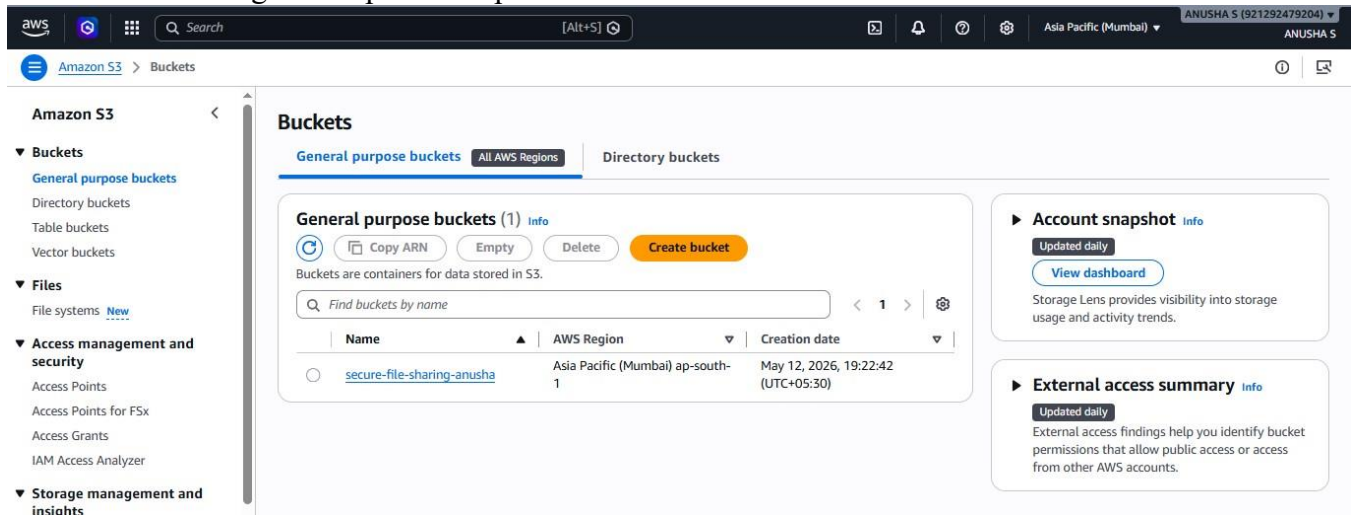
Step-by-Step Procedure

Step 1 – Create an S3 Bucket

1. Open AWS Console
2. Navigate to **S3**

3. Click **Create Bucket**
4. Enter bucket name
5. Enable **Block Public Access**
6. Create bucket

The bucket is configured as private to prevent direct file access.



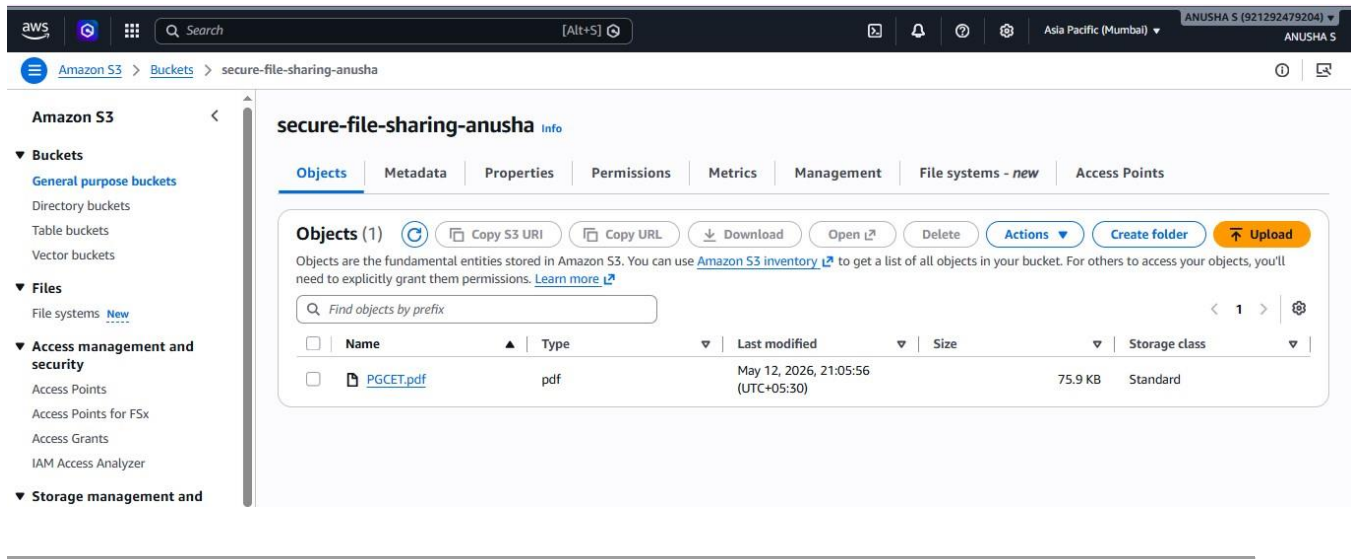
Step 2 – Upload Files to S3

Upload files that will be shared securely.

Example:

test.pdf

These files remain private and cannot be accessed directly.



Step 3 – Create CloudFront Distribution

1. Open CloudFront
2. Click **Create Distribution**
3. Select the S3 bucket as the origin
4. Configure Origin Access Control (OAC)
5. Enable HTTPS redirection
6. Enable caching settings
7. Create the distribution

CloudFront acts as the secure gateway between users and S3.

CloudFront > **Distributions** > E2BATVIOX8STTD

secure-file-sharing Free plan View metrics

Details

Distribution domain name d1ssw8rfvkasth.cloudfront.net	Billing Free plan (\$0/month) Manage plan	ARN arn:aws:cloudfront::921292479:204:distribution/E2BATVIOX8STTD	Last modified May 13, 2026 at 3:00:24 PM UTC
--	---	---	--

General | Security | Origins | Behaviors | Error pages | Invalidations | Logging | Tags

Settings Edit

Name secure-file-sharing	Alternate domain names Add domain	Standard logging Available with the Pro plan
Description -		Default root object -
Price class Use all edge locations (best performance)		
Supported HTTP versions HTTP/2, HTTP/1.1, HTTP/1.0		

CloudFront > **Distributions** > E2BATVIOX8STTD

secure-file-sharing Free plan View metrics

Details

Distribution domain name d1ssw8rfvkasth.cloudfront.net	Billing Free plan (\$0/month) Manage plan	ARN arn:aws:cloudfront::921292479:204:distribution/E2BATVIOX8STTD	Last modified May 13, 2026 at 3:00:24 PM UTC
--	---	---	--

General | Security | Origins | Behaviors | Error pages | **Invalidations** | Logging | Tags

Invalidations (2) View details Copy to new Create invalidation

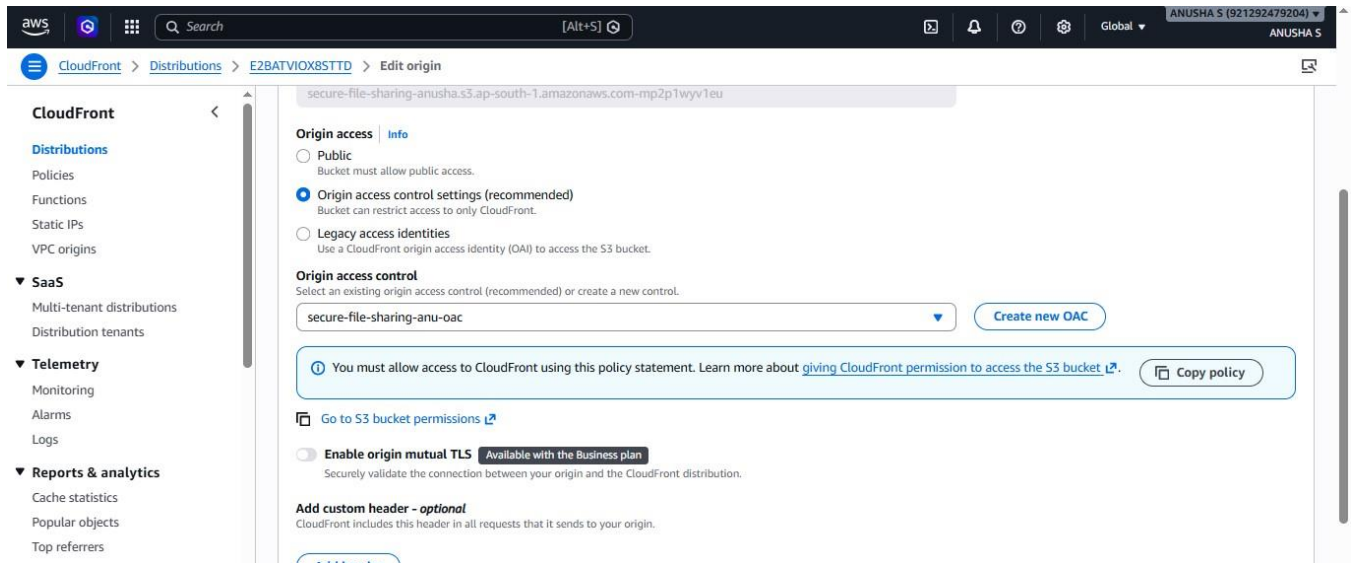
Invalidation ID	Status	Date created
IF2A3U31M5C7G9UAAQOG5U0X4G	Completed	May 12, 2026 at 3:46:13 PM UTC

Step 4 – Configure Origin Access Control

Create a new OAC and attach it to the origin.

Purpose:

Only CloudFront can access the S3 bucket.



Step 5 – Update S3 Bucket Policy

Allow CloudFront to read objects from the bucket.

Example permission:

Allow CloudFront service principal to perform:

s3:GetObject

on the bucket objects.

Step 6 – Create Public Key and Key Group

Generate RSA key pair using OpenSSL.

Commands used:

```
openssl genrsa -out private_key.pem 2048
```

```
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

Upload the **public key** to CloudFront and create a **key group**.

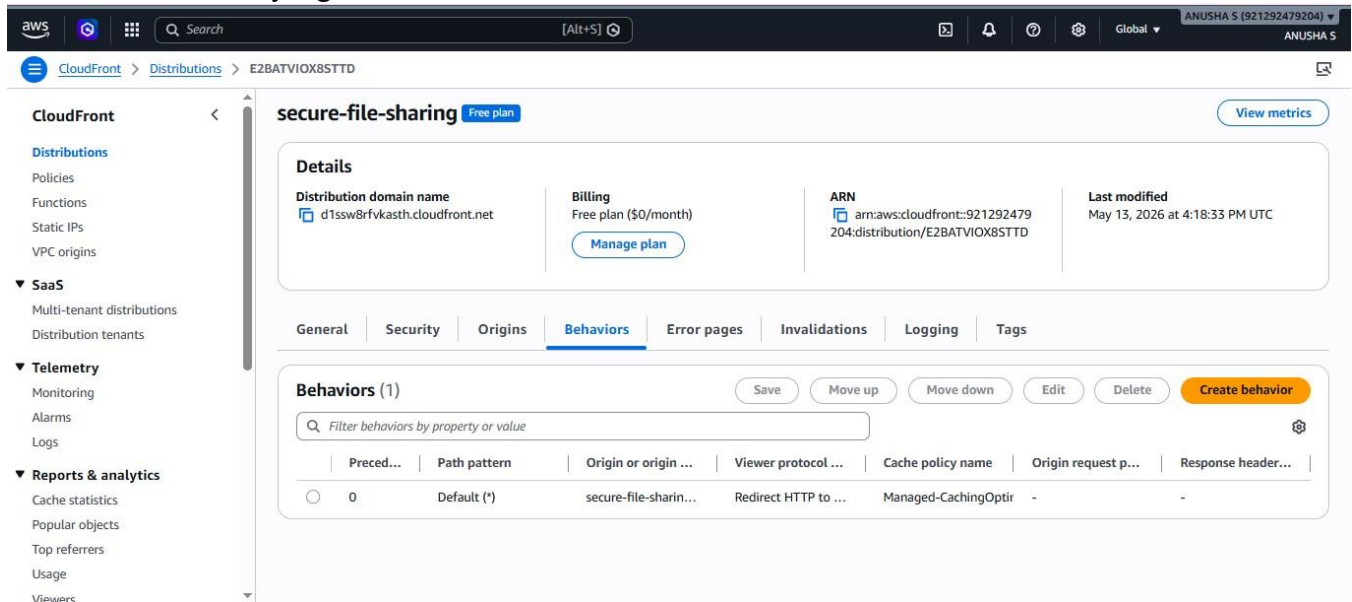
Step 7 – Configure CloudFront Behavior

Enable:

Restrict Viewer Access = YES

Attach the **trusted key group**.

This ensures that only signed URLs can access the content.



Step 8 – Create Lambda Function

Create a Python Lambda function to generate signed URLs.

The function:

- Reads the requested file name
- Generates a signed CloudFront URL
- Sets an expiration time

Step 9 – Add RSA Layer to Lambda

Install required dependency:

rsa library

Create a Lambda layer and attach it to the function.

The screenshot shows the AWS Lambda console interface. The top navigation bar includes the AWS logo, a search bar, and the user's name 'ANUSHA S'. The main content area is titled 'generateSignedURL' and has tabs for 'Code', 'Test', 'Monitor', 'Configuration', 'Aliases', and 'Versions'. The 'Configuration' tab is selected, showing a 'General configuration' section with an 'Edit' button. The configuration details include:

Property	Value
Description	-
Memory	128 MB
Ephemeral storage	512 MB
Timeout	0 min 3 sec
SnapStart	None

On the left, a sidebar lists various configuration options: General configuration, Triggers, Permissions, Destinations, Function URL, Environment variables, Tags, VPC, RDS databases, Monitoring and operations tools, and Concurrency and recursion detection. The bottom of the console shows a footer with 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for 2026.

Step 10 – Generate Signed URL

Lambda returns a signed URL such as:

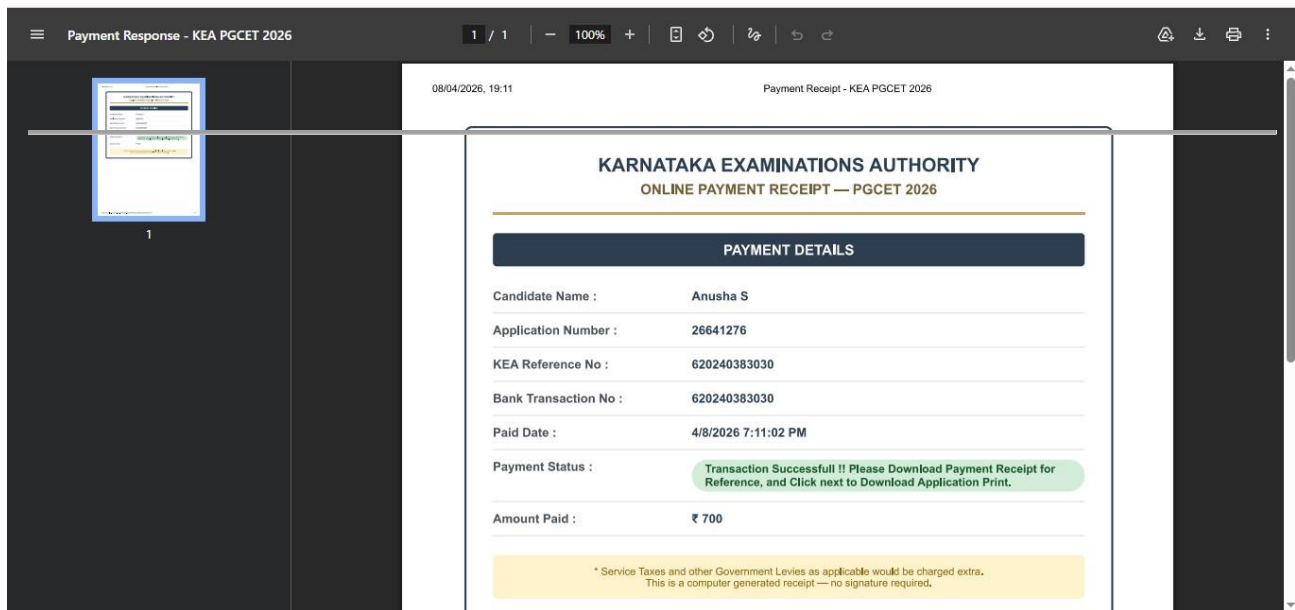
`https://cloudfront-domain/test.pdf?Expires=...&Signature=...&Key-Pair-Id=...`

This URL allows temporary access to the file.

The screenshot shows the AWS Lambda console interface for the 'generate-signed-url' function. The top navigation bar is the same as the previous screenshot. The main content area has tabs for 'Code', 'Test', 'Monitor', 'Configuration', 'Aliases', and 'Versions'. The 'Test' tab is selected, showing a 'Diagram' view with a single function node labeled 'generate-signed-url'. A green notification banner at the top states 'The test event "test1" was successfully saved.' Below the diagram, there are buttons for '+ Add trigger' and '+ Add destination'. On the right, a 'Description' panel shows details about the function, including its ARN and URL. At the bottom, a green box displays the execution result for the 'test1' event, indicating it was successful. The details of the execution show a JSON response with a signed URL.

```

{
  "statusCode": 200,
  "body": "{\n  \"signed_url\": \"https://d330m16cg1k5c6.cloudfront.net/test.pdf?Expires=1771408561&Signature=elNth1dKUHsAQ125qz5F-3j8tztv18gR72LYbmbR-oAfzpAxeN90Gt6P71Fwt797s16AvTF3kgppXaoVUCK13KxYxYdbM1a57f4zDoxzur6CnZn1FBUy0GEn1F7j19XW-C4zD216j-Pdto81MtPaWkt-m-o-K0hK2ajx-ktp8uvccpd5rt6ZgkdAwq-XNRfuovKyr6bJ5TJtoFFYwcoIG-gZUpCb3p2rqZBLQ620F0m09QLG1ZHLQMSz5HzAR82LPpkNhp7vVF10z6aanHz2kY4SH1P9j-GFcTeyN2J01K5M-WlghsR5XfV0vUE1-oKQCtyz4e8CTupC4Ja__&Key-Pair-Id=K261TH8FIX8YE\" }\n"
}
```



Troubleshooting

Issue 1 – Access Denied

Method

Check CloudFront and S3 permissions.

Solution

Update the bucket policy and ensure correct Origin Access Control configuration.

Issue 2 – Missing Key Pair ID

Method

Verify CloudFront key group configuration.

Solution

Ensure the correct Key Pair ID is included in the signed URL.

Issue 3 – Invalid Public Key

Method

Verify RSA key format.

Solution

Ensure the key is in proper PEM format with BEGIN and END markers.

Issue 4 – Lambda Error "File name required"

Method

Check Lambda event input.

Solution

Provide the file name parameter in the request.

Example:

file=test.pdf

Conclusion

This project demonstrates how AWS services can be combined to create a **secure file-sharing system**. By integrating Amazon S3, CloudFront, and Lambda, the system ensures that files remain private while still being delivered efficiently to authorized users.

The use of signed URLs adds an additional layer of security by providing temporary access to files. This architecture is widely used in production environments for secure document distribution, media streaming, and enterprise file access systems.

The solution is scalable, secure, and cost-effective due to its serverless architecture.